works well for a variety of scenarios. For more in-depth information check out the documentation on the Quartz website (quartz-scheduler.org/documentation/).

The next listing shows an excerpt from my code that defines a Quartz job that calls one of our services. Take a look at grails-app/jobs/org/cwhd/measure/PopulatorJob.groovy[2] for the full code.

[2] github.com/cwhd/measurementor/blob/master/grails-app/jobs/org/cwhd/measure/PopulatorJob.groovy

Listing B.4. Basic job to run our services

```
class DataFetchingJob {

def jiraDataService
static triggers = {
  simple name: 'jobTrig', startDelay: 60000, repeatInterval: 100000
}

def execute() {
  def result = "unknown"
  try {
    def startDateTime = new Date()
    jiraDataService.getData(0, 100, "ACOE")
    stashDataService.getAll()
    def doneDateTime = new Date()
    def difference = doneDateTime.getTime()-startDateTime.getTime()
    def minutesDiff = TimeUnit.MILLISECONDS.toMinutes(difference)
    result = "success in $difference ms"
    println "------------------------------------------------"
    println "ALL DONE IN ~$minutesDiff minutes"
    println "------------------------------------------------"
  } catch (Exception ex) {
    result = "FAIL: $ex.message"
  }
  JobHistory history=new JobHistory(jobDate:new Date(),jobResult:result)
  history.save(failOnError: true)
}
```

Declares the service to call from the job.

Sets up the trigger to run.

The method called when the job is run; calls the service from here.

Times the method.

Saves the job history for reference.

This class could be only a few lines of code because we only need to define the timer and call the services that get the data. The rest of the code times how long it takes to execute, handles errors, and saves the details of the job into the `JobHistory` domain class.

The `JobHistory` class is used to remember when jobs were executed and the result. If for some reason they failed, then the next time the job runs it